SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSIGNMENT

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Scenario Based Assignment
Weightage:	20% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	Purushothaman.V	**Reg. No.:**	192511064
Section / Batch:	CSE	**Date:**	23/10/2026

Code of Conduct

I Purushothaman.V / 192511064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Purushothaman.V

Instructions

- Read each scenario carefully before answering (Off-Class Assessment)
- Answer all five scenario-based questions. Each question carries and is designed to be answered in about 12 minutes.
- Identify the key issues in each scenario, apply the relevant AI technique, and justify every decision with clear reasoning.
- Questions are drawn from Informed Search (Greedy, A*), Local Search and Optimization, Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Marks will be awarded using the Scenario Based Assignment rubric: understanding of scenario, identification of key issues, application of concepts, decision making and clarity of response.
- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions	**Q1**
2. Local Search and Online Search	Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms), online search agent	**Q2, Q3**
3. Constraint Satisfaction Problem	Variables, domains, constraints, backtracking search, constraint propagation (forward checking)	**Q4**
4. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	**Q5**

Rubrics for Scenario Based Assignment

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Scenario	**20%**	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	**20%**	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	**25%**	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	**20%**	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	**15%**	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed search strategies (Greedy best-first search, A, heuristic functions) | 12 minutes*

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic: roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to:

- A heuristic estimate (straight-line distance) to the destination.
- Partial real-time updates about blocked paths.
- Variable movement costs depending on terrain and weather.

Due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

(a) Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

(b) Explain how A* would handle the same situation, including how it balances cost and heuristic.

(c) Analyze the impact of heuristic accuracy on both algorithms.

(d) Discuss how dynamic changes (blocked paths) affect search performance.

(e) Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

Question 1: Answer

(a) Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

Greedy Best-First Search selects the node that appears closest to the destination using a heuristic function $h(n)$. In this scenario, the drone uses straight-line distance to estimate how close each location is to the medicine-delivery destination.

The algorithm repeatedly selects the location with the smallest heuristic value and continues until it reaches the destination.

Strengths:

1. It can find a route quickly when time is critical.
2. It uses heuristic information to guide the search toward the destination.
3. It may explore fewer nodes than uninformed search algorithms.

Weaknesses:

1. It ignores the actual cost already incurred to reach a location.
2. A geographically short route may have flooded paths, strong winds or difficult terrain.
3. Incomplete real-time information may cause the drone to select a blocked route.
4. It does not guarantee the minimum-cost or safest route.

Therefore, Greedy Best-First Search is fast but may not be reliable enough for emergency medicine delivery in an uncertain environment.

(b) Explain how A* would handle the same situation, including how it balances cost and heuristic.

A* Search evaluates each node using the function:

$$f(n) = g(n) + h(n)$$

Where:

- $f(n)$= estimated total cost of the route through node n.
- $g(n)$= actual cost from the starting point to node n.
- $h(n)$= estimated cost from node $n$to the destination.

The drone selects the node with the lowest $f(n)$. Unlike Greedy Best-First Search, A* considers both the cost already incurred and the estimated remaining cost.

For example, a short route may have a high travel cost because of flooding or adverse weather. A* can select a slightly longer route if its total estimated cost is lower.

With non-negative movement costs and an admissible heuristic, A* can find an optimal route under the standard search assumptions. In a changing environment, the drone must update its route information and replan when necessary.

(c) Analyze the impact of heuristic accuracy on both algorithms.

A heuristic estimates the remaining cost from the current location to the destination. Its accuracy significantly affects search performance.

In Greedy Best-First Search, the heuristic directly determines which node is selected. If the straight-line distance does not reflect flooding, wind or terrain conditions, the drone may choose a short but expensive route.

In A* Search, the heuristic is combined with the actual path cost $g(n)$. Therefore, A* considers both the distance estimate and the cost already incurred.

An admissible heuristic never overestimates the actual minimum remaining cost. This property helps A* preserve optimality under the standard assumptions. An inaccurate or overestimating heuristic may reduce efficiency or remove the optimality guarantee.

Thus, a suitable heuristic should consider the environment as accurately as possible without overestimating the remaining cost when optimality is required.

(d) Discuss how dynamic changes (blocked paths) affect search performance.

Dynamic changes, such as flooded roads or blocked flight paths, can make a previously selected route unusable.

When a path becomes blocked, the drone must update its environment model, mark the path as unavailable and search for an alternative route.

These changes can cause:

1. Repeated searches and increased computation.
2. Delays in medicine delivery.
3. Increased battery consumption due to route changes.
4. Difficulty finding a safe route when real-time information is incomplete.

The drone should use updated sensor information, validate routes before following them and maintain an alternative safe route. Repeated or incremental A* search can help the system respond to environmental changes.

(e) Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality and safety.

*Recommendation: A Search with real-time updates and replanning.**

A* is more suitable because it considers both actual travel cost and estimated remaining cost. Its cost function can include travel time, terrain difficulty, weather conditions, battery consumption and safety risk.

Greedy Best-First Search may find a route quickly, but it ignores the accumulated cost and may select an unsafe path.

A* can provide an optimal route for the current search model when its standard assumptions are satisfied. However, because the flood environment changes dynamically, the drone must continuously update its information and replan when paths become blocked.

Therefore, A* combined with real-time information and safety checks provides a better balance of route quality, efficiency and safety.

Question 2: Smart City Traffic Signal Optimization

Topic: Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms) | 12 minutes

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize:

- Total waiting time.
- Fuel consumption.
- Traffic congestion.

The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Tasks:

(a) Formulate this as an optimization problem and explain why traditional search is inefficient.

(b) Explain how Hill Climbing would work in this scenario and identify its limitations.

(c) Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

(d) Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

(e) Recommend the best approach and justify your choice based on scalability and adaptability.

Question 2: Answer

(a) Formulate this as an optimization problem and explain why traditional search is inefficient.

The problem can be formulated as an optimization problem in which the AI system searches for the best traffic signal timings across hundreds of intersections.

The objective is to minimize:

1. Total vehicle waiting time.
2. Fuel consumption.
3. Traffic congestion.

A weighted objective function can be written as:

$$J = \alpha W + \beta F + \gamma C$$

Where:

- W= total waiting time.
- F= fuel consumption.
- C= traffic congestion.
- α, β, γ= weights representing the importance of each factor.

The decision variables include green-light duration, cycle length, signal phase order and coordination between intersections.

Traditional exhaustive search is inefficient because the number of possible timing combinations increases rapidly with the number of intersections. Evaluating every possible configuration requires substantial computation, and traffic patterns continue to change during the search.

Therefore, local search and optimization algorithms are more practical for this problem.

(b) Explain how Hill Climbing would work in this scenario and identify its limitations.

Hill Climbing starts with an initial traffic signal configuration and evaluates neighbouring configurations.

The algorithm selects a neighbouring configuration that improves the objective function. Since the objective is to minimize traffic cost, it moves toward a configuration with lower waiting time, fuel consumption and congestion.

The process continues until no neighbouring configuration provides an improvement.

Advantages:

1. Simple to implement.

2. Requires relatively little memory.
3. Can quickly improve an existing traffic signal configuration.

Limitations:

1. It can become trapped in a local optimum.
2. It may struggle with plateaus and ridges.
3. Its performance depends on the initial configuration.
4. Improving one intersection may worsen traffic at nearby intersections.

Therefore, basic Hill Climbing may not find the best overall traffic signal configuration.

(c) Discuss issues like local maxima, plateaus and ridges with real-world interpretation.

1. Local optimum

A local optimum is a solution that cannot be improved by considering nearby solutions, even though a better solution may exist elsewhere.

For example, increasing the green-light duration at one intersection may reduce its queue but increase congestion at the next intersection. The algorithm may stop at a configuration that is locally good but globally poor.

Because this problem minimizes cost, the relevant local optimum is generally a **local minimum**. A local maximum would apply if the algorithm instead maximized a performance score.

2. Plateau

A plateau is an area in the search space where several neighbouring configurations have similar performance.

For example, changing a traffic signal by a few seconds may produce almost no difference in total waiting time. The algorithm receives little information about which direction to explore.

3. Ridge

A ridge is a narrow path of improvement that may require coordinated changes across several variables.

For example, improving traffic flow may require adjusting green-light durations at several connected intersections together. Changing only one signal at a time may fail to discover the improvement.

These problems limit the effectiveness of basic Hill Climbing.

(d) Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

Simulated Annealing

Simulated Annealing sometimes accepts a worse solution to escape a local optimum. At a high temperature, the algorithm is more willing to accept worse configurations. As the temperature decreases, it becomes more selective.

This allows the algorithm to explore a wider range of solutions before gradually concentrating on better configurations.

Genetic Algorithm

A Genetic Algorithm maintains a population of candidate traffic signal configurations. Each configuration is evaluated using the objective function.

The algorithm uses:

1. **Selection:** Selects better-performing configurations.
2. **Crossover:** Combines parts of two candidate configurations.
3. **Mutation:** Introduces small random changes.
4. **Evaluation:** Measures the performance of the new configurations.

By exploring several configurations simultaneously, a Genetic Algorithm can discover improvements that a single Hill Climbing search may miss.

Both methods can overcome some limitations of basic Hill Climbing, although neither guarantees the global optimum in every practical run.

(e) Recommend the best approach and justify your choice based on scalability and adaptability.

Recommendation: Genetic Algorithm with local refinement and continuous traffic feedback.

A Genetic Algorithm is suitable for optimizing traffic signals across hundreds of intersections because it evaluates a population of candidate configurations and explores different combinations of timings.

It can optimize multiple objectives, including waiting time, fuel consumption and congestion, while respecting safety constraints.

For better performance, the system can use a Genetic Algorithm to generate coordinated traffic plans and then apply local search to refine them.

The system should periodically update its plans using current traffic measurements. It must also enforce pedestrian safety, minimum green times, safe signal transitions and emergency-vehicle priorities.

This approach offers a practical balance between scalability, adaptability and solution quality.

Question 3: Autonomous Rover Exploring Mars

Topic: Online search agent | 12 minutes

An autonomous rover is deployed on Mars to explore unknown terrain. It must:

- Navigate safely to collect samples.
- Avoid hazards (craters, rocks).
- Operate with limited sensing capability.
- Make decisions without a complete map.

The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

(a) Explain why this problem requires an online search agent instead of offline search.

(b) Analyze the challenges of partial observability and dynamic environments.

(c) Describe how the rover builds and updates its internal model.

(d) Compare online search with uninformed and informed search strategies.

(e) Justify the most suitable approach considering uncertainty, adaptability, and safety.

Question 3: Answer

(a) Explain why this problem requires an online search agent instead of offline search.

An online search agent is required because the Mars rover operates in an unknown environment without a complete map.

An offline search agent plans using a model available before execution. However, the rover cannot know every obstacle, crater or terrain condition in advance.

The online search agent follows a continuous process:

1. Observe the environment using sensors.
2. Select an action based on current knowledge.
3. Move through the terrain.
4. Collect new information.
5. Update its internal model and plan the next action.

For example, if the rover discovers a crater along its planned route, it can update its map and search for an alternative path.

Therefore, online search is more suitable because it combines exploration, learning and decision-making.

(b) Analyze the challenges of partial observability and dynamic environments.

The rover faces several challenges due to partial observability and uncertainty.

Partial observability:

- Sensors cannot reveal the entire terrain at once.
- Rocks or craters may remain hidden until the rover approaches them.
- Sensor readings may contain errors.
- The rover may have incomplete information about slopes and surface conditions.

Dynamic and uncertain conditions:

- Newly discovered obstacles may block the planned route.
- Dust or environmental conditions may affect sensing.
- Communication delays can limit remote assistance.
- Battery and computing resources are limited.

These challenges make it difficult to create a perfect map before exploration. The rover must make decisions using incomplete information and prioritize safe movement.

(c) Describe how the rover builds and updates its internal model.

The rover maintains an internal representation of the terrain, including known locations, unexplored areas and detected hazards.

The process is as follows:

1. **Perception:** Collect data using cameras, range sensors and position estimates.
2. **Mapping:** Identify obstacles, safe paths and unknown regions.
3. **State estimation:** Estimate the rover's current position and orientation.
4. **Model update:** Incorporate newly discovered terrain information.
5. **Planning:** Select a suitable route toward the sample-collection target.
6. **Replanning:** Change the route when a hazard or unexpected condition is detected.

For example, if the rover detects loose soil, it can increase the estimated cost of travelling through that region and choose a safer alternative.

Maintaining an updated internal model allows the rover to use previous observations when the current percept alone is insufficient.

(d) Compare online search with uninformed and informed search strategies.

Approach	Description	Strengths	Limitations
Offline search	Plans using a model before execution.	Can produce a complete plan when the map is reliable.	A plan may become invalid when the environment is unknown.
Uninformed search	Searches without heuristic guidance, such as BFS or DFS.	Does not require a heuristic.	May explore many irrelevant locations.
Informed search	Uses a heuristic to guide the search toward a goal.	Can reduce unnecessary exploration.	The heuristic may be misleading when hazards are unknown.
Online search	Interleaves actions, observations and replanning.	Adapts to newly discovered information.	Requires repeated decisions and may incur additional computation.

Online search is particularly useful for the rover because it must learn about the terrain while moving through it.

(e) Justify the most suitable approach considering uncertainty, adaptability and safety.

Recommendation: Online, model-based search with informed, risk-aware planning.

The rover should maintain an internal map, use heuristic information to move toward sample locations and update its knowledge whenever new observations are received.

The route cost should consider distance, slope, obstacle risk and energy consumption rather than distance alone.

The rover should also:

- Avoid known hazards.
- Replan when unexpected obstacles are detected.
- Prefer safe routes when uncertainty is high.
- Maintain a fallback route when possible.
- Stop or retreat when safe navigation cannot be established.

This approach is more adaptable than a fixed offline plan and helps the rover explore unknown terrain while maintaining safety.

Question 4: University Exam Timetable Generation

Topic: Constraint satisfaction problem (backtracking search, constraint propagation, forward checking) | 12 minutes

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints:

- No student should have overlapping exams.
- Limited number of exam halls.
- Certain subjects must be scheduled before others.
- Faculty availability constraints.
- Special accommodations for some students.

The complexity increases as the number of courses and students grows.

Tasks:

(a) Formulate the problem as a Constraint Satisfaction Problem (CSP): clearly define the variables, domains, and constraints with explanation.

(b) Explain how backtracking search works in this scenario.

(c) Discuss how constraint propagation (for example, forward checking) improves efficiency.

(d) Analyze real-world challenges and justify the best strategy for scalability.

Question 4: Answer

(a) Formulate the problem as a Constraint Satisfaction Problem (CSP): clearly define the variables, domains and constraints with explanation.

A Constraint Satisfaction Problem consists of three main components: **variables, domains and constraints**.

1. Variables

For each course, define variables such as:

- Time(Course) — the examination time slot.
- Hall(Course) — the examination hall.
- Invigilator(Course) — the assigned faculty member, where required.

2. Domains

Domains contain the possible values that can be assigned to each variable.

For example:

- Available examination time slots.
- Available halls with sufficient seating capacity.
- Faculty members available for invigilation.

3. Constraints

The timetable must satisfy the following constraints:

a. Student-conflict constraint: Two courses with common students cannot be scheduled at the same time.

$$Time(A) \neq Time(B)$$

b. Hall-capacity constraint: The selected hall must have sufficient capacity for the students taking the examination.

c. Hall-availability constraint: A hall cannot be assigned to two examinations taking place simultaneously.

d. Precedence constraint: If one subject must be conducted before another, their time slots must follow the required order.

e. Faculty-availability constraint: Faculty members must be available, and one invigilator cannot supervise two simultaneous examinations.

f. Special-accommodation constraint: Students requiring accessibility arrangements or other approved accommodations must receive suitable facilities and scheduling arrangements.

The objective is to find a complete assignment that satisfies all hard constraints. If several valid timetables exist, the system can select one that reduces student waiting periods and distributes examinations more evenly.

(b) Explain how backtracking search works in this scenario.

Backtracking Search assigns values to variables one at a time while checking whether each assignment satisfies the constraints.

The steps are:

1. Select an unassigned course.
2. Choose a possible time slot and examination hall.
3. Check whether the assignment violates any constraint.
4. If the assignment is valid, continue with the next course.
5. If no valid assignment exists, return to the previous course.
6. Remove the previous assignment and try another possibility.
7. Continue until a valid timetable is found or all possibilities are exhausted.

For example, if Course A is assigned to Slot 1, Course B cannot be assigned to Slot 1 if students are enrolled in both courses.

If every available slot for Course B violates a constraint, the algorithm backtracks and changes an earlier assignment.

Using heuristics such as selecting the course with the fewest remaining valid time slots can reduce the search effort.

(c) Discuss how constraint propagation (for example, forward checking) improves efficiency.

Constraint propagation reduces the search space by removing values that cannot satisfy the constraints.

Forward checking is performed after assigning a value to a variable. It examines the domains of unassigned variables and removes incompatible values.

For example, if Course A is assigned to Slot 1:

1. Slot 1 is removed from the domain of every course that shares students with Course A.
2. A hall occupied by Course A in Slot 1 is removed from the available hall choices for other examinations in that slot.
3. The domains of affected courses are checked.
4. If any course has no remaining valid option, the algorithm backtracks immediately.

Forward checking detects conflicts early instead of allowing the algorithm to continue with an assignment that cannot lead to a valid timetable.

When combined with backtracking and suitable variable ordering, it improves efficiency by reducing unnecessary exploration.

(d) Analyze real-world challenges and justify the best strategy for scalability.

Real-world timetable generation involves several challenges:

1. Large numbers of courses and students.
2. Limited examination halls and seating capacity.
3. Courses with overlapping student registrations.
4. Faculty availability and invigilation requirements.
5. Different examination durations and laboratory requirements.
6. Special accommodations for certain students.
7. Last-minute changes in registrations or faculty availability.

As the number of variables increases, the search space grows significantly.

Recommended strategy: Use a CSP solver combining backtracking, forward checking, constraint propagation and variable-ordering heuristics.

The system should prioritize highly constrained courses, select values that leave more options for other courses, and use a conflict graph to identify courses that share students.

For larger problems, the timetable can be divided into manageable groups by day, campus or course category, followed by a global validation step.

Hard constraints must always be satisfied, while preferences such as minimizing gaps between examinations can be treated as optimization objectives.

This approach provides a scalable method for generating valid timetables while accommodating changing university requirements.

Question 5: Strategic Game AI for Real-Time Decision Making

Topic: Game playing: adversarial search (minimax, alpha-beta pruning, evaluation functions) | 12 minutes

A gaming company is developing an AI opponent for a real-time strategy game. The AI must:

- Compete against human players.
- Make decisions within strict time limits.
- Evaluate complex game states with multiple possible moves.

The decision tree is extremely large, making computation expensive.

Tasks:

(a) Explain how the Minimax algorithm models this problem.

(b) Analyze the computational challenges of large game trees.

(c) Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

(d) Design an evaluation function and justify the factors considered.

(e) Recommend a strategy for balancing optimality and real-time performance.

Question 5: Answer

(a) Explain how the Minimax algorithm models this problem.

Minimax is an adversarial search algorithm used in competitive games. It models the game as a search tree in which nodes represent game states and edges represent possible moves.

There are two types of decision nodes:

- **MAX:** Represents the AI player, which tries to maximize its utility.
- **MIN:** Represents the opponent, which tries to minimize the AI player's utility.

The algorithm evaluates terminal states and propagates their values upward through the search tree.

At a MAX node:

$$V(s) = \max_{a} V\big(Result(s, a)\big)$$

At a MIN node:

$$V(s) = \min_{a} V\big(Result(s, a)\big)$$

The AI selects the move with the best minimax value, assuming that the opponent also plays optimally.

Minimax is most directly applicable to deterministic, turn-taking, two-player, zero-sum games with well-defined rules.

(b) Analyze the computational challenges of large game trees.

The main challenge is the large number of possible moves and resulting game states.

If the average branching factor is $b$and the search depth is d, the time complexity of exhaustive Minimax is:

$$O(b^d)$$

As the branching factor and search depth increase, the number of nodes grows exponentially.

The major challenges are:

1. High computational cost.
2. Limited time for decision-making.
3. Large numbers of possible game states.
4. Difficulty evaluating states beyond the search depth.
5. Memory and processing limitations.
6. The risk of missing important long-term consequences when using a shallow search.

Therefore, searching every possible game state is often impractical in real-time strategy games.

(c) Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

Alpha-Beta pruning improves Minimax by eliminating branches that cannot affect the final decision.

It maintains two values:

- **Alpha (α):** The best value MAX can guarantee so far.
- **Beta (β):** The best value MIN can guarantee so far.

Whenever:

$$\alpha \geq \beta$$

the remaining branches at that point can be pruned because they cannot improve the decision for the relevant ancestor.

Example:

Suppose MAX has already found a move with a value of 5. Therefore, MAX has an existing alternative worth 5.

While examining another move, MIN finds an outcome worth 3. Since MIN can force the value down to 3, MAX would prefer the earlier move worth 5.

The remaining branches under that alternative do not need to be examined if they cannot change the decision. Alpha-Beta pruning eliminates this unnecessary computation.

Advantages:

1. Reduces the number of nodes evaluated.
2. Allows deeper search within the same time limit.
3. Produces the same minimax decision as exhaustive Minimax when applied correctly.
4. Works particularly well when promising moves are examined first.

With ideal move ordering, Alpha-Beta pruning can reduce the time complexity to approximately:

$$O\left(b^{d/2}\right)$$

In the worst case, however, its complexity remains $O(b^d)$.

Thus, Alpha-Beta pruning improves search efficiency without changing the Minimax result.

(d) Design an evaluation function and justify the factors considered.

An evaluation function estimates how favourable a non-terminal game state is for the AI player.

One possible evaluation function is:

$$Eval(s) = w_1M + w_2R + w_3O + w_4T + w_5P - w_6D$$

Where:

- M= material and resources.
- R= resource production.
- O= objective control.
- T= tactical position.
- P= progress and initiative.
- D= damage and risk.
- $w_1, w_2, \dots, w_6$= weights assigned to the factors.

Explanation of factors:

1. Material and resources: Measures the number of units, buildings, currency or other resources available. Greater resources can provide more strategic options.

2. Resource production: Measures the ability to generate resources over time. Higher production may support long-term growth.

3. Objective control: Evaluates control of important locations and progress toward the victory conditions.

4. Tactical position: Considers unit positioning, attack opportunities, defensive strength and threats.

5. Progress and initiative: Measures map control and the ability to act before the opponent.

6. Damage and risk: Accounts for vulnerable units, exposed buildings and possible losses. Greater risk reduces the evaluation score.

The weights should be selected according to the game's rules and tested against different opponents.

The evaluation function must consistently represent the AI player's perspective. Terminal wins and losses should receive values larger in magnitude than ordinary heuristic scores so that a forced win is preferred over a merely favourable intermediate position.

(e) Recommend a strategy for balancing optimality and real-time performance.

Recommendation: Depth-limited Minimax with Alpha-Beta pruning, iterative deepening and heuristic evaluation.

The AI should search the most promising moves first to improve pruning efficiency. Instead of searching the entire game tree, it should evaluate states up to a selected depth using the evaluation function.

Recommended improvements:

1. **Alpha-Beta pruning:** Removes branches that cannot affect the final decision.
2. **Move ordering:** Examines promising moves first.
3. **Iterative deepening:** Searches depth 1, then depth 2, depth 3 and so on.

4. **Transposition table:** Stores previously evaluated states to avoid repeating work.
5. **Time management:** Stops the search before the decision deadline.
6. **Heuristic evaluation:** Estimates the value of states that are not terminal.

Iterative deepening is particularly useful because the AI can retain the best move from the last completed search if the time limit is reached.

This strategy balances computational efficiency and decision quality. It does not guarantee a globally optimal move when the search is depth-limited, but it can provide strong decisions within real-time constraints.